KUIS PRAKTIK 1

PENGUJIAN PERANGKAT LUNAK



Disusun oleh:

RIZKYA DAFA NUR MUHAMMAD

2405097

POLITEKNIK NEGERI INDRAMAYU

JURUSAN TEKNIK INFORMATIKA

PROGRAM STUDI REKAYASA PERANGKAT LUNAK

2026

A. Gambaran Praktikum dan Adaptasi Stack

Praktikum ini melakukan kontainerisasi aplikasi PolyLife menggunakan Docker dan Docker Compose. PolyLife memakai Laravel 12, PHP, dan MySQL. Layanan web menjalankan Nginx serta PHP-FPM; database berada pada container terpisah.

Implementasi mengadaptasi skenario Node.js/Express dan PostgreSQL pada soal menjadi stack proyek yang digunakan. Adaptasi ini perlu persetujuan pengajar dan tidak dinyatakan sebagai pemenuhan literal seluruh ketentuan image pada soal.

Aspek	**Implementasi PolyLife**
Runtime aplikasi	php:8.2-fpm-alpine; Nginx; user www-data
Database	mysql:8.4; database polylife_quiz; user polylife
Port	Container 3000; host 8088 saat pengujian
Penyimpanan	db_data -> /var/lib/mysql
Jaringan	app_net; driver bridge; prefix project polylife-quiz

B. Dockerfile dan Docker Compose

Dockerfile menggunakan multi-stage build. composer.json/composer.lock disalin sebelum instalasi dependency PHP; package.json/package-lock.json disalin sebelum npm ci. Tahap Node.js 22 hanya membangun aset Vite. Runtime memakai WORKDIR /app, USER www-data, dan EXPOSE 3000. Build tool frontend tidak dibawa ke image runtime.

Compose memuat service web dan db. web menunggu healthcheck db melalui condition: service_healthy. Healthcheck MySQL menjalankan SELECT 1 memakai user aplikasi. Database tidak membuka port ke host; volume dan network terisolasi dari stack pengembangan.

Default port host pada Compose adalah 8080. Bukti pengujian memakai APP_PORT=8088 karena port 8080 sudah digunakan stack lain. APP_ENV=local dan APP_DEBUG=false dipakai untuk HTTP lokal agar aplikasi tidak memaksakan URL aset HTTPS. Konfigurasi lengkap terdapat dalam:

Dockerfile

```
FROM php:8.2-fpm-alpine AS php-base

WORKDIR /app

RUN apk add --no-cache nginx supervisor curl libpng libjpeg-turbo libwebp freetype libzip oniguruma \

    && apk add --no-cache --virtual .build-deps $PHPIZE_DEPS libpng-dev libjpeg-turbo-dev libwebp-dev freetype-dev libzip-dev oniguruma-dev \

    && docker-php-ext-configure gd --with-webp --with-jpeg --with-freetype \

    && docker-php-ext-install -j2 pdo_mysql mbstring pcntl bcmath gd opcache zip \

    && apk del .build-deps
```

```
FROM php-base AS dependencies
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --no-interaction --prefer-dist --no-progress
COPY . .
RUN mkdir -p storage/framework/cache/data storage/framework/sessions storage/framework/views storage/logs bootstrap/cache \
    && composer dump-autoload --no-dev --classmap-authoritative --no-scripts \
    && php artisan package:discover --ansi

FROM node:22-alpine AS assets
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY resources ./resources
COPY public ./public
COPY vite.config.js tailwind.config.js postcss.config.js ./
RUN npm run build

FROM php-base AS runtime
ENV APP_ENV=production APP_DEBUG=false LOG_CHANNEL=stderr
COPY --from=dependencies --chown=www-data:www-data /app /app
COPY --from=assets --chown=www-data:www-data /app/public/build /app/public/build
COPY ["docs/quiz ppl/container/nginx.conf", "/etc/nginx/nginx.conf"]
COPY ["docs/quiz ppl/container/supervisord.conf", "/etc/supervisord.conf"]
COPY ["docs/quiz ppl/container/php-fpm.conf", "/usr/local/etc/php-fpm.d/zz-quiz.conf"]
RUN mkdir -p storage/framework/cache/data storage/framework/sessions storage/framework/views storage/logs bootstrap/cache \
    && chown -R www-data:www-data storage bootstrap/cache
USER www-data
EXPOSE 3000
```

```
CMD ["/usr/bin/supervisord", "-c", "/etc/supervisord.conf"]
```

docker-compose.yml

```
name: polylife-quiz
services:
  web:
    image: polylife-quiz:local
    build:
      context: ../..
      dockerfile: docs/quiz ppl/Dockerfile
    init: true
    ports:
      - '127.0.0.1:${APP_PORT:-8080}:3000'
    environment:
      APP_NAME: PolyLife
      APP_ENV: local
      APP_DEBUG: 'false'
      APP_KEY: ${APP_KEY:?Set APP_KEY in .env}
      APP_URL: http://localhost:${APP_PORT:-8080}
      LOG_CHANNEL: stderr
      DB_CONNECTION: mysql
      DB_HOST: db
      DB_PORT: 3306
      DB_DATABASE: polylife_quiz
      DB_USERNAME: polylife
      DB_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD in .env}
      CACHE_STORE: database
      SESSION_DRIVER: database
      QUEUE_CONNECTION: database
      MAIL_MAILER: log
      HASH_DRIVER: bcrypt
```

```
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ['CMD', 'curl', '--fail', '--silent', 'http://127.0.0.1:3000/up']
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s
    networks: [app_net]
  db:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: polylife_quiz
      MYSQL_USER: polylife
      MYSQL_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD in .env}
                      MYSQL_ROOT_PASSWORD:      ${DB_ROOT_PASSWORD:?Set
DB_ROOT_PASSWORD in .env}
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
          test: ['CMD-SHELL', 'MYSQL_PWD="$${MYSQL_PASSWORD}" mysql --
protocol=TCP -h 127.0.0.1 -u "$${MYSQL_USER}" "$${MYSQL_DATABASE}" -e
"SELECT 1" >/dev/null']
      interval: 5s
      timeout: 5s
      retries: 30
      start_period: 30s
    networks: [app_net]
volumes:
  db_data:
networks:
```

```
app_net:
  driver: bridge
```

C. Urutan Perintah CLI dan Status Container

Seluruh perintah dijalankan dari folder docs/quiz ppl. Pada eksekusi pertama, migrasi Laravel dan akun demo disiapkan sebelum pengujian API.

Menjalankan stack di latar belakang

```
docker compose up -d --build --wait
docker compose exec -T web php artisan migrate --force
```

Memeriksa log service web

```
docker compose logs --tail 50 web
```

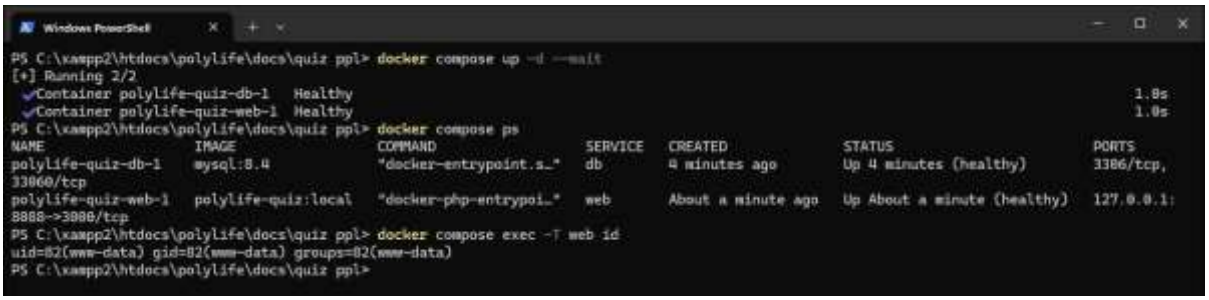
Masuk ke shell database dan menampilkan database

```
docker compose exec db sh
MYSQL_PWD="$MYSQL_PASSWORD" mysql -u "$MYSQL_USER" "$MYSQL_DATABASE"
SHOW DATABASES;
```

Ketik exit untuk keluar dari MySQL, lalu exit untuk keluar dari shell container.

Menghapus container dan network dengan mempertahankan volume

```
docker compose down
```

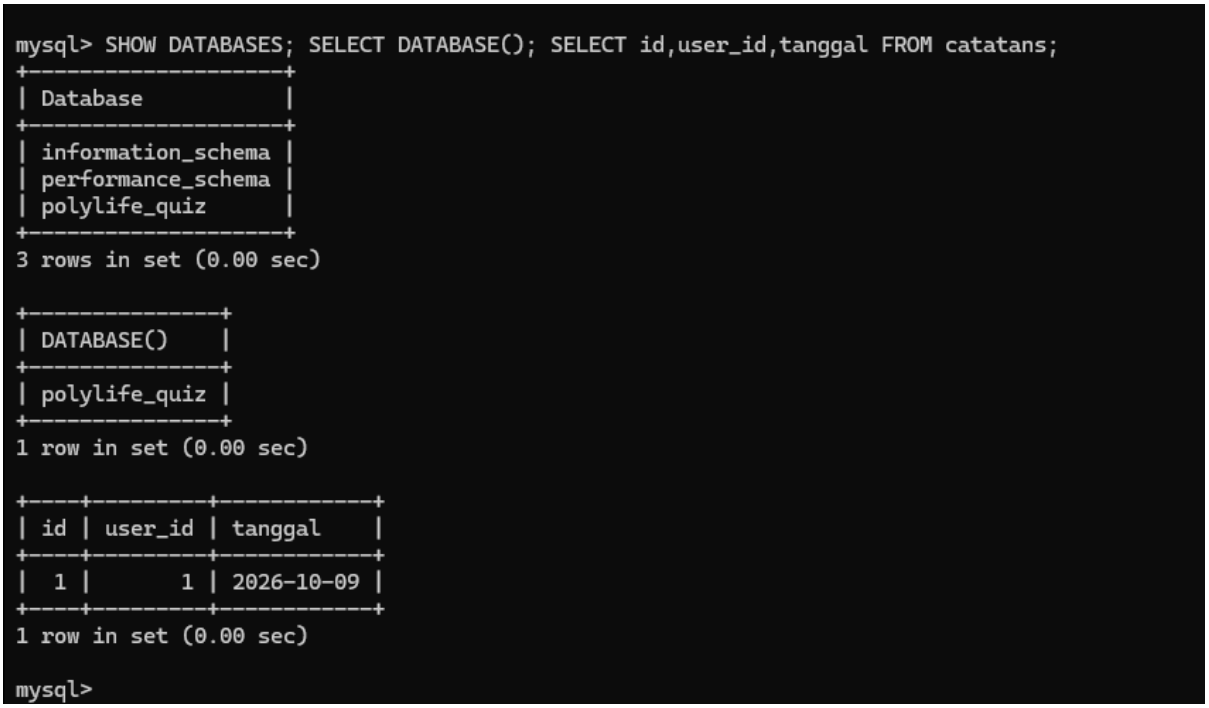


Gambar 1. Service web dan db healthy; proses aplikasi menggunakan UID 82 (www-data).

Verifikasi tambahan: docker compose ps menampilkan pemetaan 127.0.0.1:8088 ke port container 3000. Output id memastikan proses aplikasi tidak memakai root.

D. Verifikasi Database dan Log Aplikasi

Pengujian shell MySQL menampilkan database polylife_quiz serta catatan ID 1, user ID 1, tanggal 2026-10-09. Data tersebut dibuat melalui endpoint API pada stack Docker.



Gambar 2. SHOW DATABASES, database aktif, dan baris catatan pada MySQL container.

Log web menunjukkan request autentikasi dan pembacaan catatan mendapat 200; request catatan tanpa data mendapat 422. Respons API yang memuat data tersimpan serta hasil SELECT menguatkan bukti hubungan aplikasi dengan database.

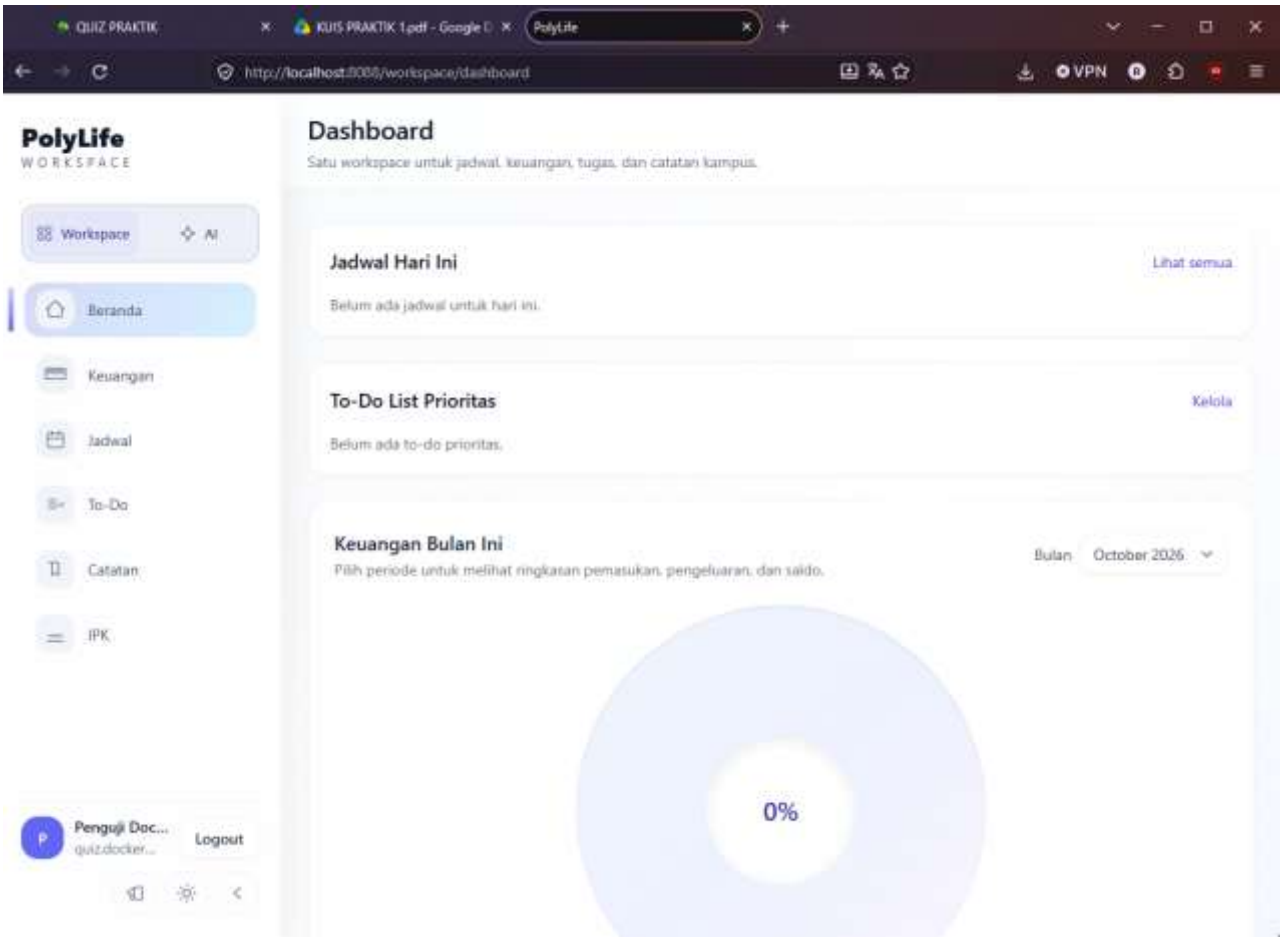


Gambar 3. Log request API pada service web di stack Docker.

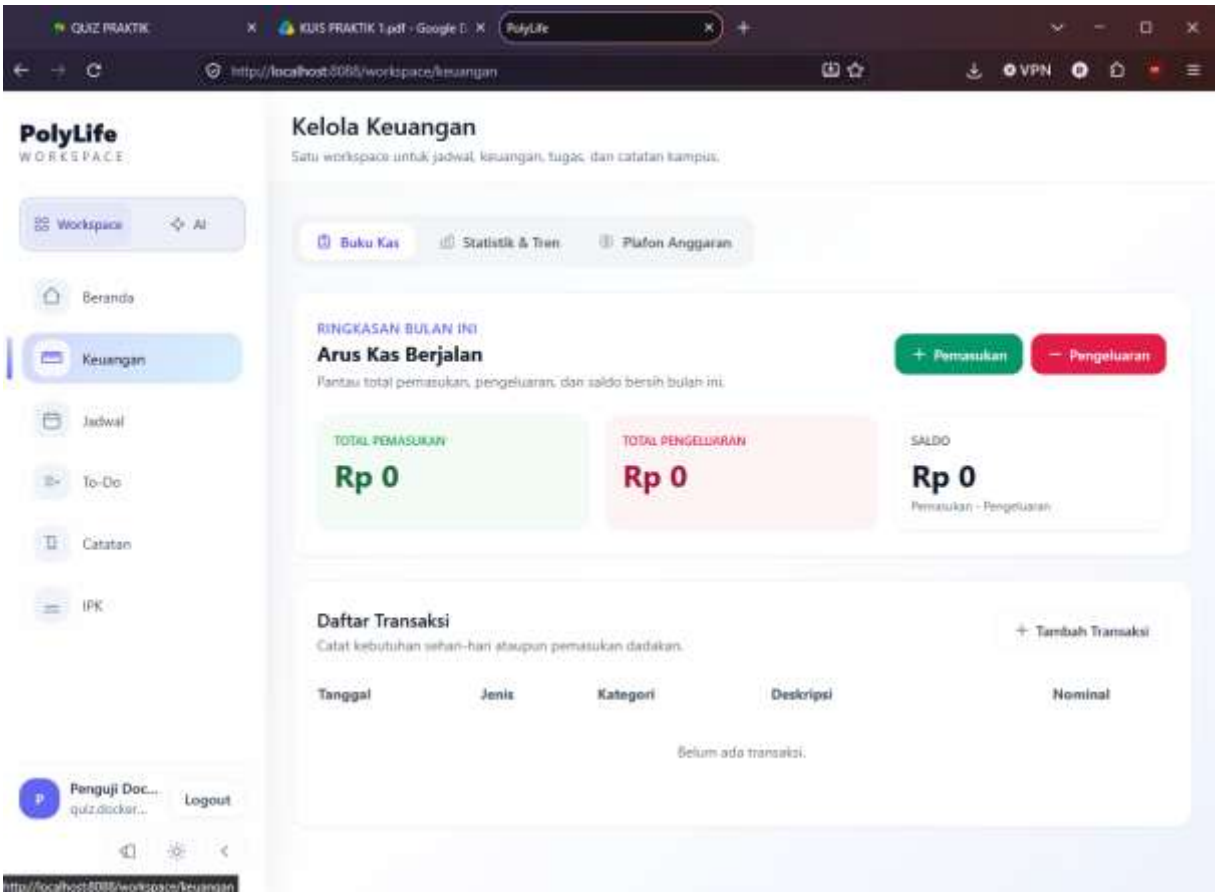
E. Pengujian Aplikasi dalam Docker



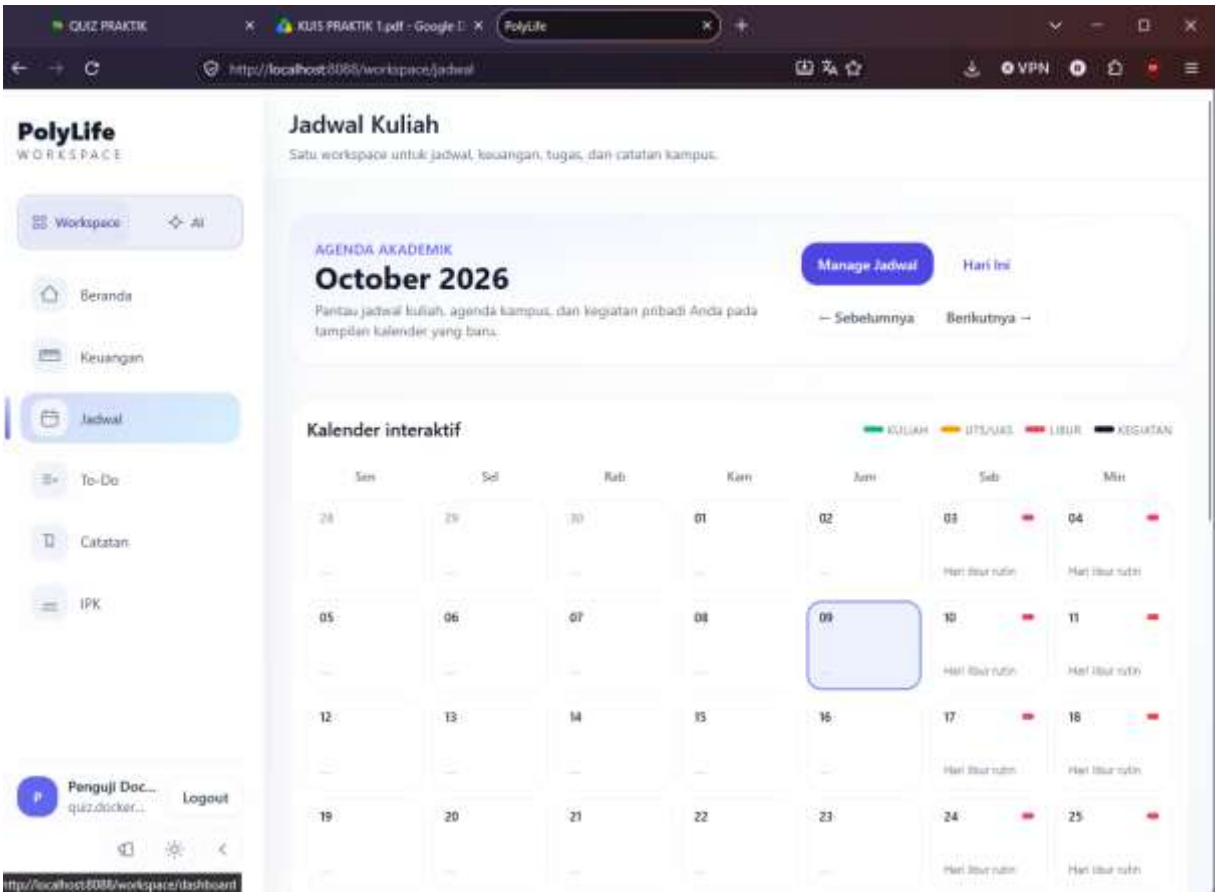
Gambar 4. Halaman PolyLife dan aset frontend berhasil dimuat pada localhost:8088.



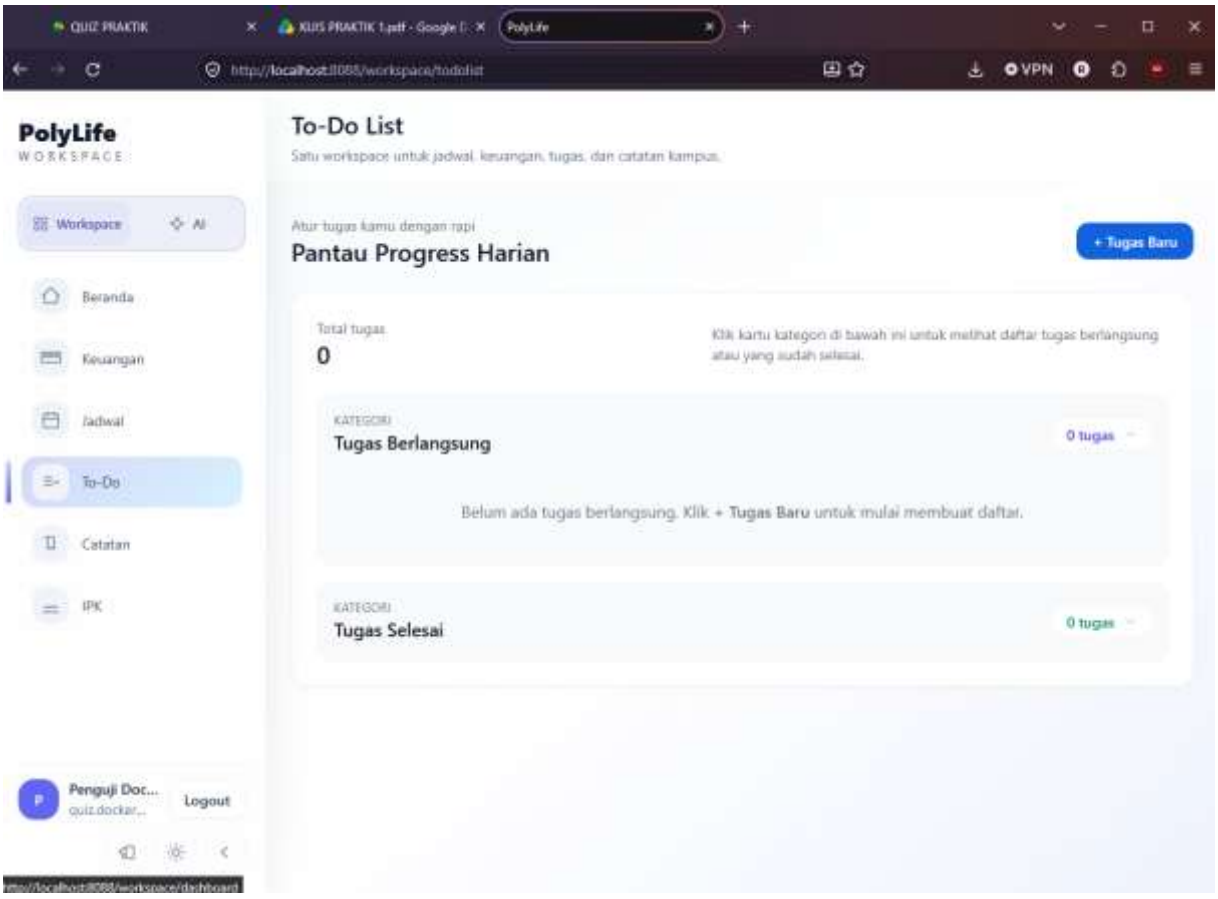
Gambar 4.1. Halaman Dashboard PolyLife dan aset frontend berhasil dimuat pada localhost:8088.



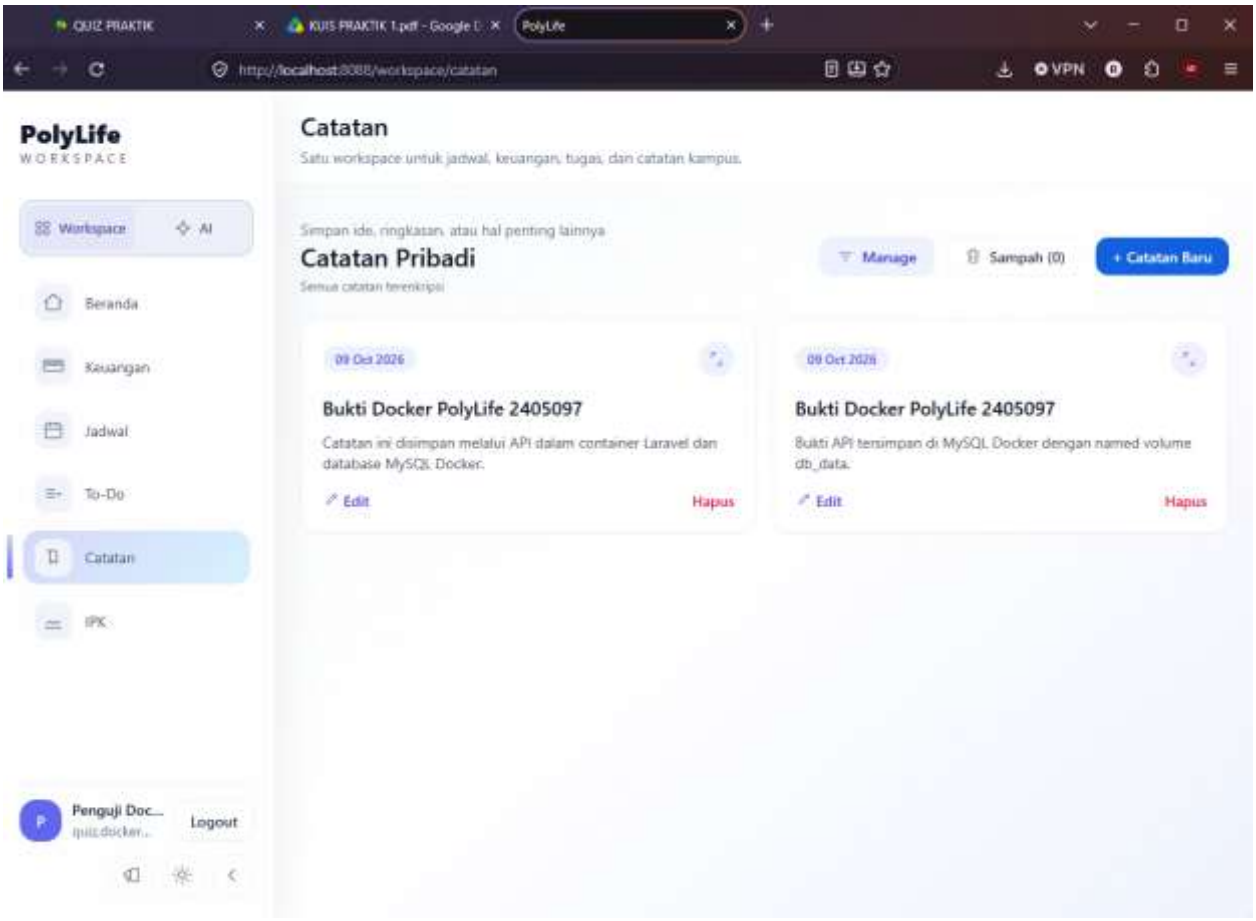
Gambar 4.3. Halaman Keuangan PolyLife dan aset frontend berhasil dimuat pada localhost:8088.



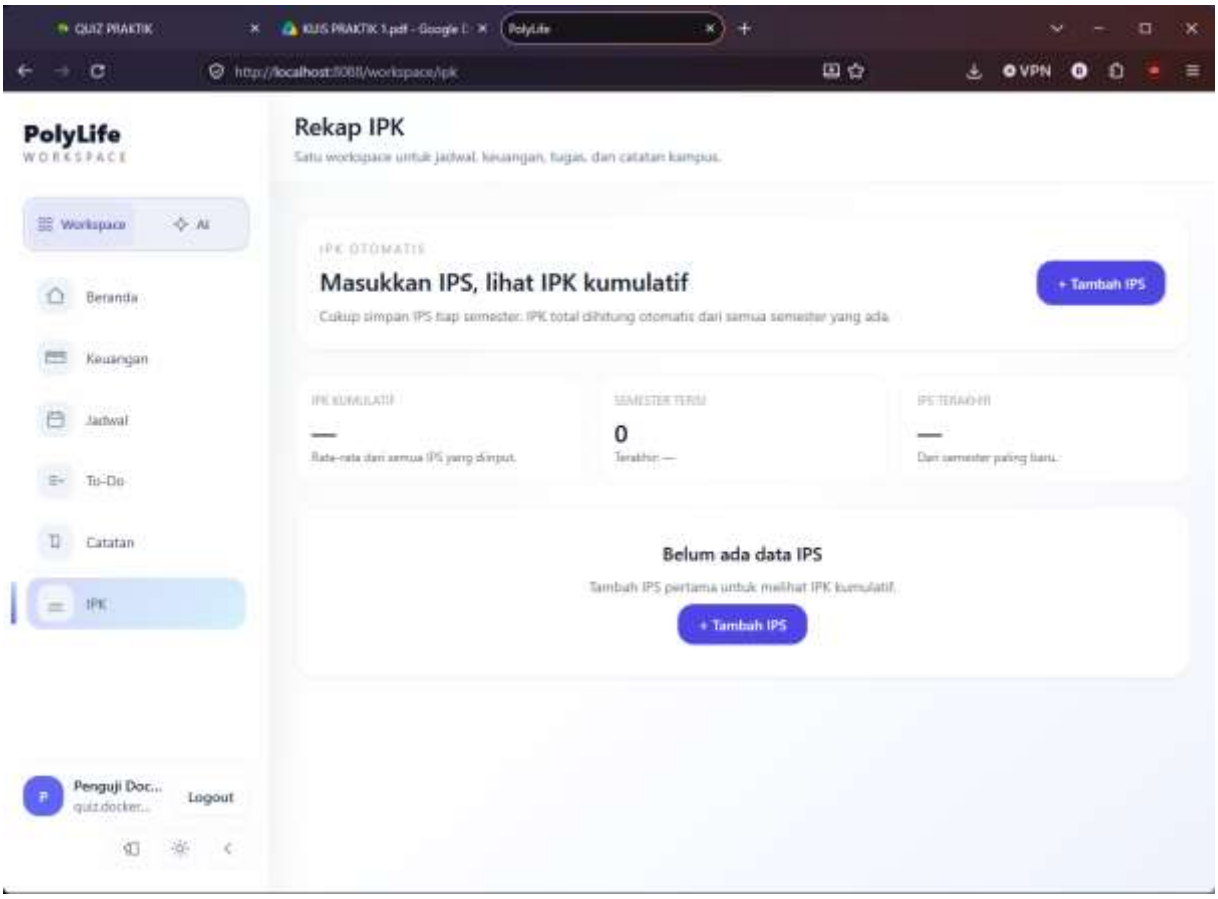
Gambar 4.3. Halaman Jadwal PolyLife dan aset frontend berhasil dimuat pada localhost:8088.



Gambar 4.4. Halaman To-Do PolyLife dan aset frontend berhasil dimuat pada localhost:8088.



Gambar 4.5. Halaman Catatan PolyLife dan aset frontend berhasil dimuat pada localhost:8088.



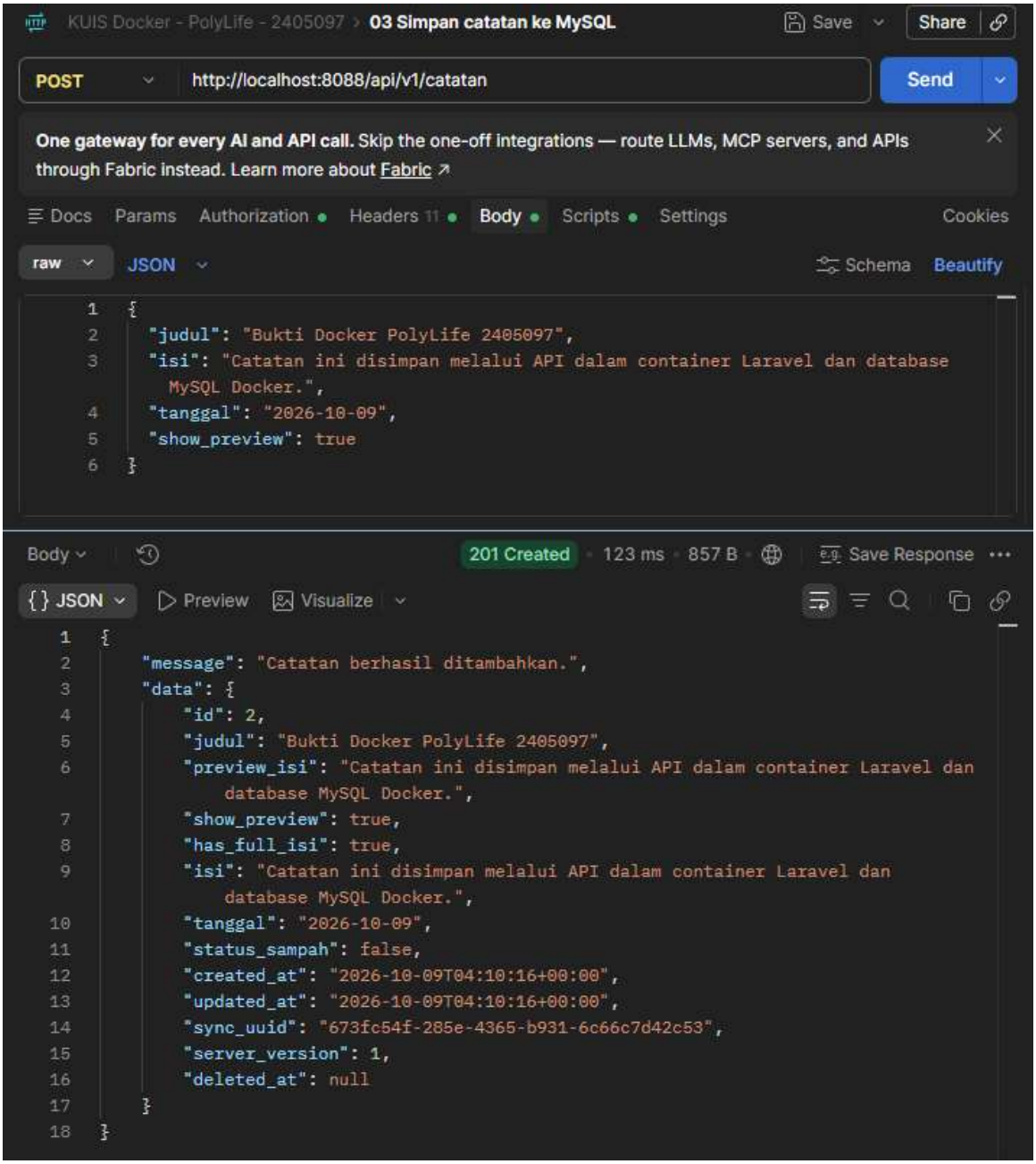
Gambar 4.6. Halaman IPK PolyLife dan aset frontend berhasil dimuat pada localhost:8088.

Pengujian API baru dijalankan melalui HTTP pada port 8088 dengan akun demo khusus database praktikum. Hasil tersimpan dalam logs/api-docker-results.json; bearer token dihapus dari bukti dan dicabut setelah pengujian.

Request	Hasil	Bukti
POST /api/v1/auth/login	200	Login berhasil
GET /api/v1/auth/me	200	Profil akun demo terbaca
POST /api/v1/catatan	201	Catatan baru ID 1 tersimpan
GET /api/v1/catatan/1	200	Judul sesuai data yang dikirim
POST /api/v1/catatan, body kosong	422	Validasi input bekerja
POST /api/v1/auth/logout	200	Token sementara dicabut

F. Pengujian Postman: Membuat Catatan

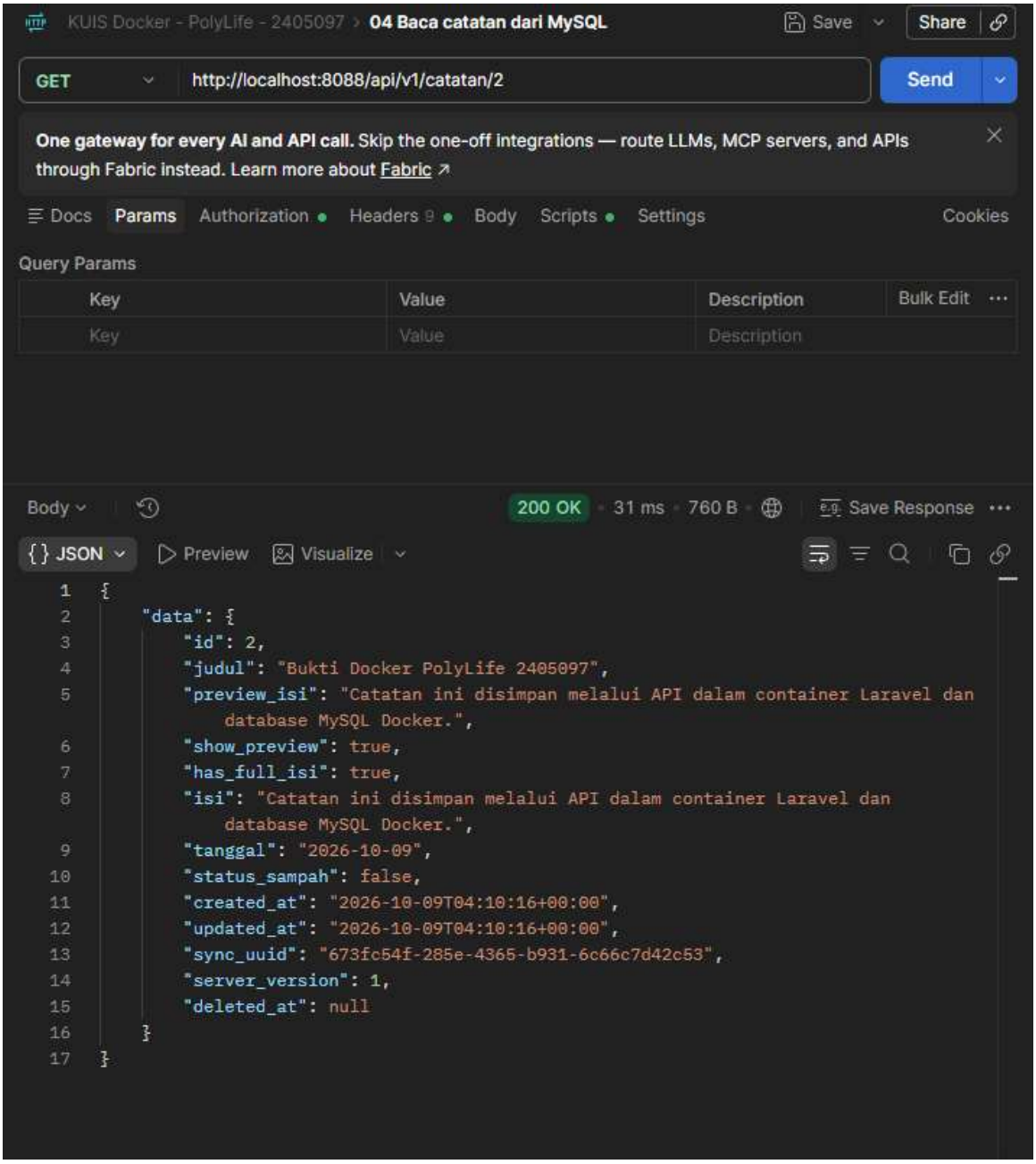
Pengujian dilakukan melalui Postman pada http://localhost:8088/api/v1/catatan. Request POST berisi judul, isi, tanggal, dan show_preview menghasilkan 201 Created. Respons memuat catatan ID 2 dan waktu pembuatan 9 Oktober 2026. Data disimpan pada MySQL dalam stack Docker.



Gambar 5. POST melalui Postman menghasilkan 201 Created dan catatan ID 2.

G. Pengujian Postman: Membaca Catatan

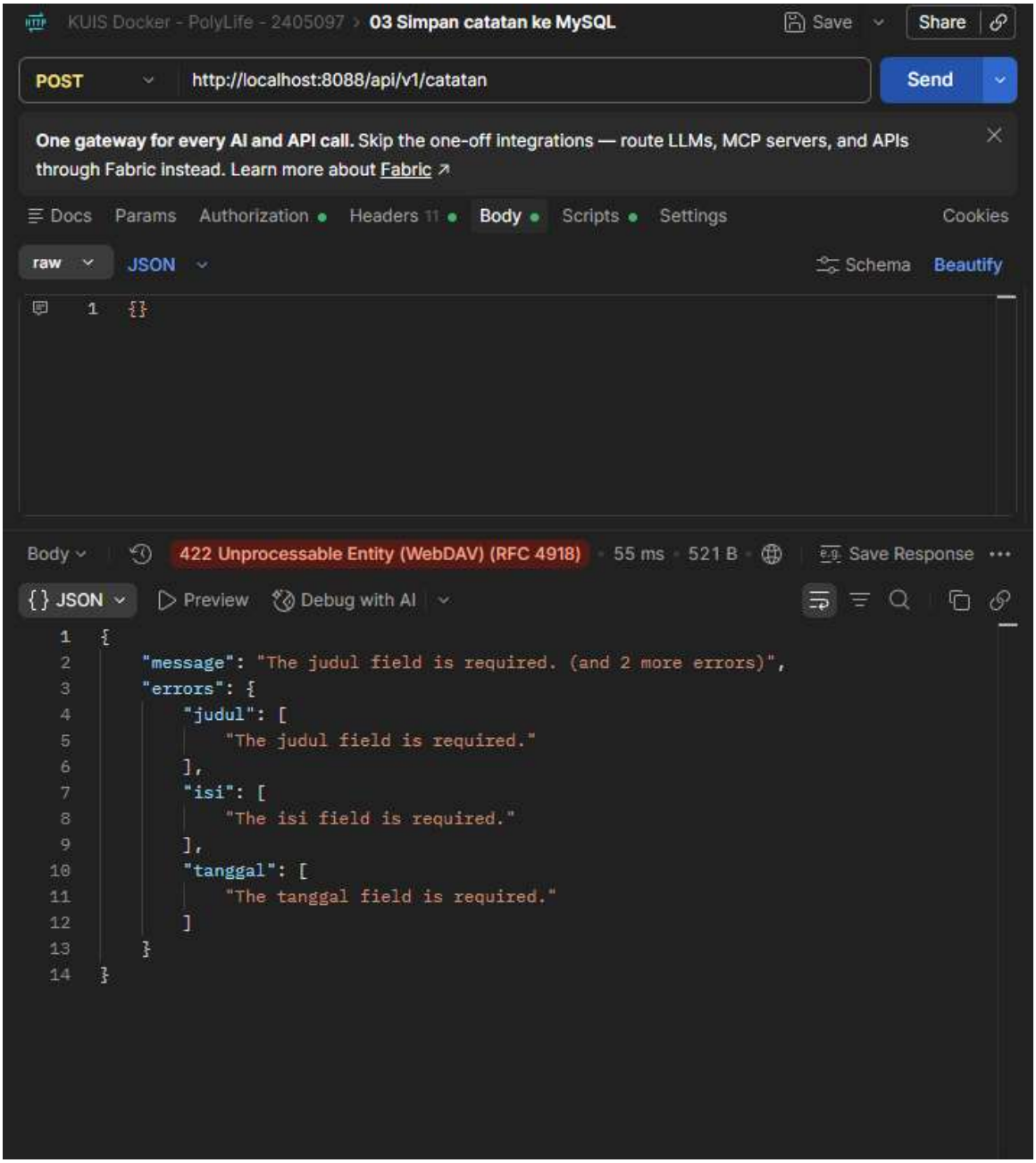
Request GET http://localhost:8088/api/v1/catatan/2 menghasilkan 200 OK. Respons memuat catatan ID 2 dengan judul dan isi sesuai request POST. Pengujian ini memastikan data yang baru disimpan dapat dibaca kembali melalui aplikasi Docker.



Gambar 6. GET detail catatan ID 2 menghasilkan 200 OK dengan data yang sesuai.

H. Pengujian Postman: Validasi Input

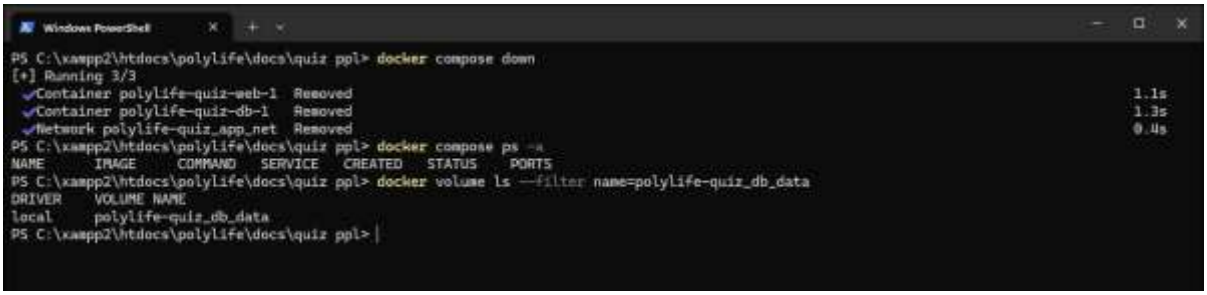
Request POST http://localhost:8088/api/v1/catatan dengan body JSON kosong ({}) ditolak dengan 422. Respons menjelaskan bahwa field judul, isi, dan tanggal wajib diisi. Validasi ini dijalankan melalui Postman pada stack Docker yang sama.



Gambar 7. Input kosong ditolak; respons menjelaskan field yang wajib diisi.

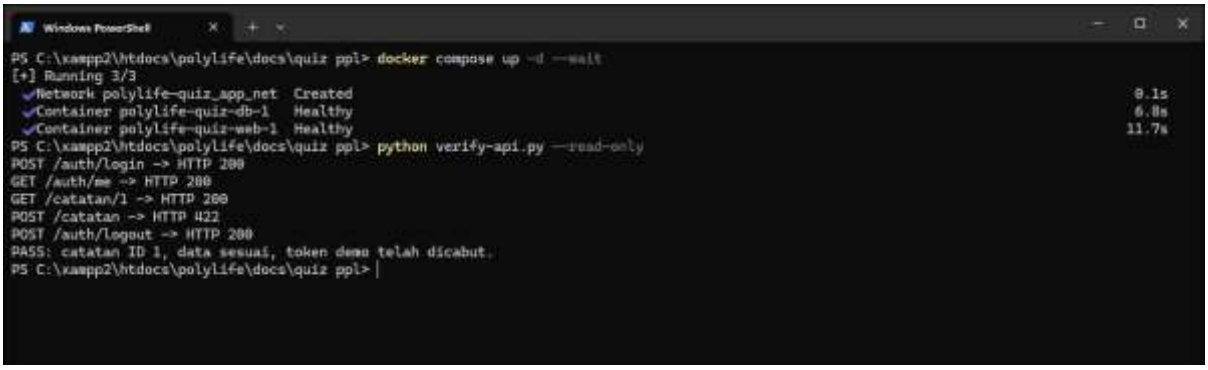
I. Persistensi Data dan Hasil Akhir

docker compose down menghapus kedua container dan network. Volume polylife-quiz_db_data tetap tercantum. Opsi -v tidak digunakan sehingga database tetap tersimpan.



Gambar 8. Container dan network dihapus; named volume tetap tersedia.

Stack kemudian dibuat ulang memakai docker compose up -d --wait. Pengujian python verify-api.py --read-only membaca catatan ID 1 dan memastikan judulnya tetap sama tanpa membuat catatan baru.



Gambar 9. Setelah container dibuat ulang, catatan ID 1 tetap terbaca dan verifikasi lulus.

Hasil: kontainerisasi HTTP/API Laravel dan MySQL berfungsi, proses aplikasi non-root, dependency database healthy, jaringan bridge terpisah, serta data bertahan saat container diganti. Stack ditinggalkan dalam keadaan aktif untuk pemeriksaan ulang. Worker AI, scheduler, email eksternal, dan push notification tidak termasuk pengujian ini.

Referensi

Docker Docs: https://docs.docker.com/compose/how-tos/startup-order/
Docker Docs: https://docs.docker.com/build/building/best-practices/